TEAM 02

AmarShohor

A community based Urban Issue Mapping Platform
for Bangladeshi Citizens

SPL 2 FINAL PRESENTATION

Team Members

Procheta Silvie
230042114

Contributions

- Interactive Map Module
- Heatmap & Geographic Visualization
- Emergency Alert
- Issue Categorization & Filtering
- Location & Place Search Module

Tasnia Farzana Nuzhat
230042124

Contributions

- Admin Panel
- City Corporation Area Management
- City Corp Portal (officials)
- Volunteer Teams
- Role Management (Super Admin, City Admins, City Corporations)

Aksan Anan Ria
230042154

Contributions

- Feed
- Issue Reporting
- Community Validation (votes, confirmations, duplicate checks)
- Moderation and Spam Control
- User Login and Authentication



Why AmarShohor

CENTRALISED PLATFORM FOR ISSUE REPORTING

GEOSPATIAL VISIBILITY OF REPORTED ISSUES

EMERGENCY ALERTS WITH LIVE EVIDENCE

COMMUNITY VALIDATION, NOT ADMIN APPROVAL

FIXED BY VOLUNTEERS OR THE CITY CORPORATION

Key Features of AmarShohor

Issue Reporting

- Photo or video, category and size
- GPS location or a manual pin
- Duplicate check within 50 m

Community Feed & Validation System

- Weighted upvotes and on-site confirmations
- "Not real?" flags hide fakes
- Comment and follow

Emergency Alerts

- 999 shown first, then neighbours within 1 km are alerted
- Live in-app photos stamped with time and GPS

Map View

- Hexagon, heat and pin views
- Only validated, open issues add heat

Volunteer Tasks & Teams

- Accept alone or grow a team of up to 10
- Earn reputation and climb the leaderboard

User Roles

Citizen

- Report issues with evidence
- Vote, flag and confirm on site
- Confirm or dispute a fix

Volunteer

- Lead or join tasks near their home area
- Fix with live on-site photos
- Earn reputation

City Corp. Official

- Accepts escalated issues in their own area
- Posts progress and submits the fix

Area Admin

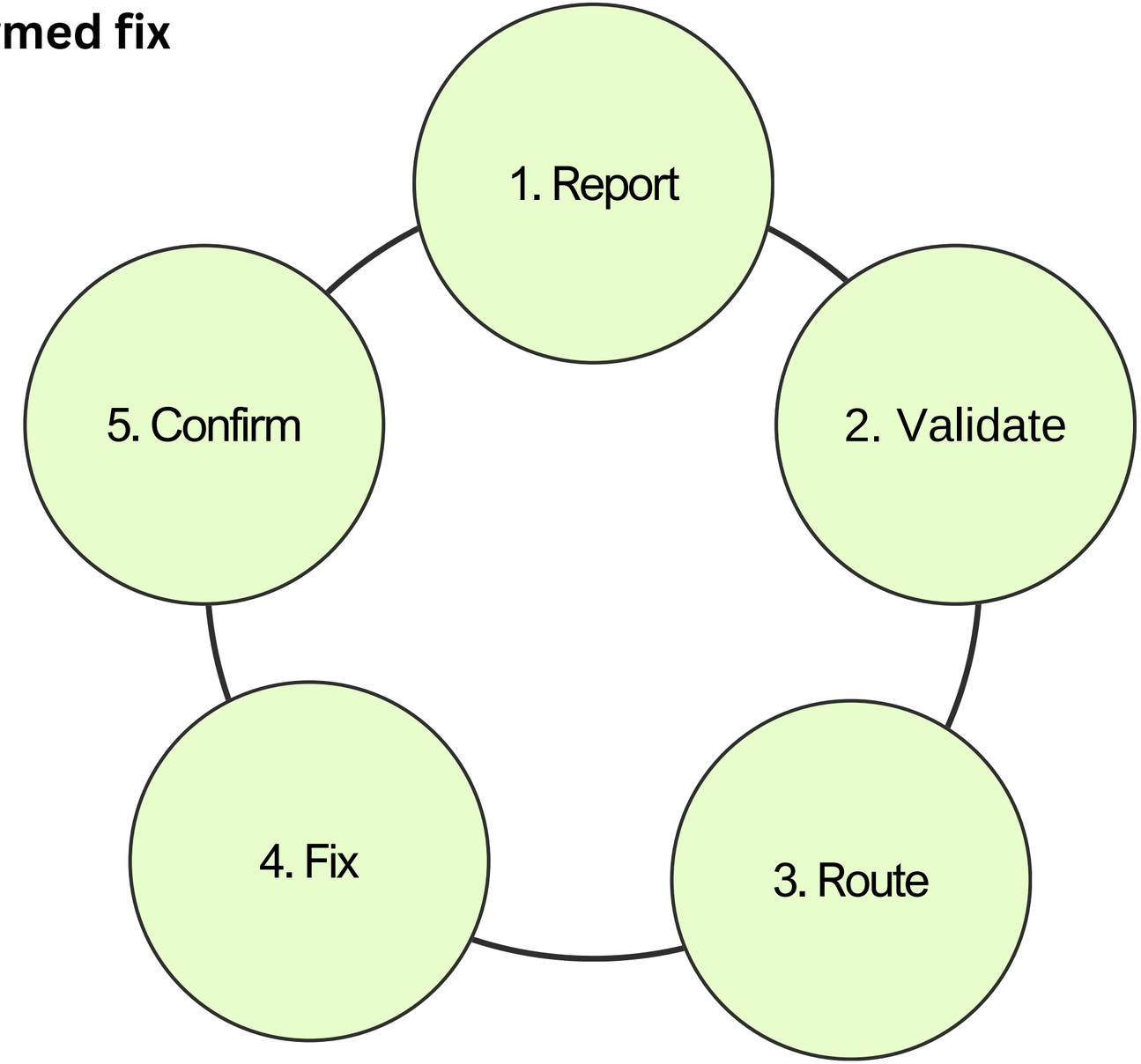
- Decides unclear cases in one City Corporation
- Verifies officials and emergency evidence

Super Admin

- Sets up City Corporations and categories
- Appoints area admins and oversees every area

Issue Lifecycle

From report to confirmed fix



Community Validation

WEIGHTED VOTES

ACCOUNT AGE, DISTANCE AND REPUTATION SET EACH VOTE'S WEIGHT

SEVERITY THRESHOLDS

CRITICAL 3 · HIGH 5 · MEDIUM 7 · LOW 10, AT LEAST 2 SUPPORTERS

HIDING FAKE REPORTS

5+ flags that outweigh support hide a post

POSTING PAUSE

3 hidden reports in 30 days: no new reports for 7 days

Heatmap & Map

HEXAGON GRID

EACH ISSUE IN EXACTLY ONE HEXAGON, COMPUTED IN POSTGIS

HEAT FORMULA

SEVERITY × EVIDENCE × FRESHNESS

TIME FILTER

All time, last 30 or last 7 days

SEARCH & AREA SUMMARY

Drop a pin to see heat and top issues nearby

Project Structure (1/4): Repository Layout

AmarShohor/	
.github/workflows/	ci.yml: checks every push
code/	all source code
web/	FRONT END: React + TypeScript
supabase/	BACK END: the database
migrations/	78 SQL files: 33 tables, 217 functions
tests/	4 SQL scenario suites
seed.sql	demo data
doc/	course documents
srs.pdf, design.pdf	SRS and design document
final-report.pdf	final report (+ these slides)
guides/	6 team guides
README.md	build, run and test steps

Why this layout

Course layout
code/ holds everything that runs; doc/ holds the SRS, design document, slides and report.

Front end and back end apart
code/web is the React app. code/supabase holds every table, rule and permission as versioned SQL, so the whole database can be rebuilt from the repo.

Checked on every push
GitHub Actions runs lint, unit tests, the build, and the SQL scenarios on a fresh database.

Project Structure (3/4): Inside code/web/src

code/web/src/	
main.tsx	entry point
app/	routes, layout, providers
core/	Supabase client, BaseService, AppError
models/	13 domain types (Issue, Profile …)
shared/	ui, components, hooks, geo, lib
test/	shared test helpers
features/	17 feature modules
issues/	one feature, opened up:
pages/	routed pages
components/	UI for this feature
hooks/	React hooks
services/	issue.service.ts
lib/	plain logic, unit tested
map/ volunteer/ emergency/	
…	

Project Structure (4/4): Why This Structure

Why this structure

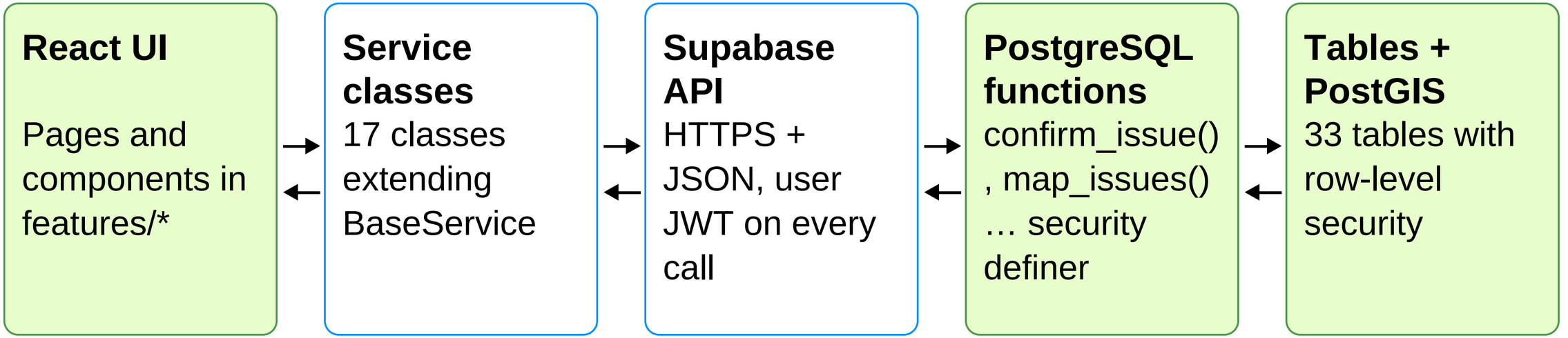
Split by feature
Each feature keeps its pages, components, hooks, services and logic in one folder.

Dependencies point one way
app → features → shared → core; every layer may use models.

Logic kept testable
Calculations live in lib/ next to their tests (67 test files).

Pages load on demand
Pages are lazy-loaded, so the first screen downloads less.

Communication Between Components (I/3): Request Path



Requests flow right
The UI calls a service, the service calls the Supabase API, and Supabase runs one of our SQL functions on the tables.

Results flow back left
JSON shaped like our models, or an error with a code that the UI can act on.

Services outside our own code

Supabase Auth
Handles sign-in and issues the JWT that every call carries.

Supabase Storage
MediaService uploads photos and gets back a storage path.

Supabase Realtime
Pushes new notification rows to the UI as they happen.

Nominatim / Photon
OpenStreetMap place search, called from PlaceSearchService.

pg_cron
Runs maintenance and stale-issue checks on a schedule.

Communication Between Components (3/3): Data & Message Format

Front end ↔ back end

data classes in code/web/src/models

```
interface Issue {   // from get_issue(), 73 fields
  id: string;  title: string
  category: string | null
  lat: number;  lng: number
  status: IssueStatus;  severity: Severity
  validation_score: number
  confirmation_count: number
  my_confirmed: boolean;  media: MediaItem[]  …
}
interface UploadedMedia {
  path: string;  type: 'image' | 'video'
}
class AppError extends Error { code: string | null }
```

Back end ↔ database

JSON message over HTTPS

```
POST /rest/v1/rpc/confirm_issue
Authorization: Bearer <JWT>
{ "p_issue": "8f3c…",
  "p_lat": 23.81,  "p_lng": 90.41,
  "p_accuracy_m": 12,  "p_note": "Still there",
  "p_media": [{ "path": "u/5c1e….jpg",
                "type": "image" }],
  "p_category": null }
→ success: empty body
→ refused: { "message": "You need to be within
…",
             "hint": "TOO_FAR" }
→ becomes AppError(code: TOO_FAR)
```

Example values shown; field names are the real ones from the code.

End-to-End Example (1/3): Confirming an Issue

1 UI

ConfirmOnSiteDialog

User taps “I see this too”. The dialog takes a GPS fix and a photo.

→

2 Service → Storage

MediaService.uploadMedia()

Photo compressed to a 1600 px JPEG and uploaded. Returns UploadedMedia {path, type}.

3 Service

IssueService.confirmIssue()

Calls this.rpc('confirm_issue', {p_issue, p_lat, p_lng, p_media …}) with the user's JWT.

→

4 Database

confirm_issue() checks

require_user(). Refuses own issue, a repeat, a closed issue, or too far away (ST_Distance).

End-to-End Example (2/3): Confirming an Issue

5 Database

confirm_issue() writes

Saves the confirmation with distance and weighted vote, logs an event, attaches the photo.

→

6 Database

recompute_issue()

Recalculates validation_score. Enough support moves the issue to 'validated'.

7 Realtime

notify(reporter)

Adds a notification row. Realtime pushes it to the reporter’s bell.

→

8 UI

Refresh

Toast says thanks. React Query refetches get_issue → the updated Issue JSON.

End-to-End Example (3/3): When a Check Fails

If a check in step 4 fails

1. The database refuses
RAISE EXCEPTION 'You need to be within … m' USING HINT = 'TOO_FAR'

2. The API returns the error
A JSON body with two fields: {message, hint}

3. BaseService converts it
It becomes AppError(code: 'TOO_FAR'), so the UI can react to the code.

4. The dialog handles it
Shows the message to the user and deletes the unused photo from Storage.

Why the code is split this way

One service class per feature
IssueService, MapService, VolunteerService … each owns one area. A method is one backend action, usually 1–5 lines.

Shared code written once
All 17 services extend BaseService: the client, error handling and media format live there. Tests inject a fake client.

Classes only where there is state
LocationService (last GPS fix), PlaceSearchService (request queue, caches), MediaService (upload retries). Components stay functions.

Layers
Pages show data, hooks fetch it, services call the back end, SQL functions hold the rules: one function per user action.

Division of Classes and Methods (2/3): Lines of Code

Lines of code

Unit	Count	Average	Maximum
TS classes	22	31.9	212 (PlaceSearchService)
TS class methods	146	4.3	35 (searchPhoton())
React components, hooks, helpers	412	25.0	382 (MapPage)
SQL functions (business logic)	217	22.1	116 (run_maintenance())

Measured with the TypeScript compiler API over code/web/src (test files excluded) and over the SQL migrations.

Division of Classes and Methods (3/3): The Largest Units

PlaceSearchService (212 lines)
Wraps two outside search APIs that share one rate-limited queue and cache.

MapPage (382 lines)
Wires the map, filters, layers and side panel; it is next in line to be split.

run_maintenance() (116 lines)
The 15-minute scheduled job: several cleanup steps in one transaction.

Technology Stack

FRONTEND

REACT 19, VITE, TYPESCRIPT, TAILWIND CSS

MAPS

LEAFLET, HEAT LAYER, MARKER CLUSTERS, OPENSTREETMAP

BACKEND

Supabase Auth, Storage, Realtime, pg_cron

DATABASE

PostgreSQL + PostGIS, 78 migrations

Technology Stack

FRONTEND

REACT 19, VITE, TYPESCRIPT, TAILWIND CSS

MAPS

LEAFLET, HEAT LAYER, MARKER CLUSTERS, OPENSTREETMAP

BACKEND

Supabase Auth, Storage, Realtime, pg_cron

DATABASE

PostgreSQL + PostGIS, 78 migrations

Testing & Quality

UNIT TESTS

67 VITEST FILES FOR PAGES, HOOKS AND LOGIC

DATABASE SCENARIOS

4 SQL SCENARIO SUITES ON A FRESH POSTGIS

CONTINUOUS INTEGRATION

GitHub Actions: lint, tests, build

EDGE CASES

Race conditions, fake GPS, duplicates

THANK YOU